

```
!pip install -q sentence-transformers chromadb langchain langchain-community langchain-huggingface
```

```
from sentence_transformers import SentenceTransformer
import chromadb
```

```
# Load embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')

print("Embedding model loaded successfully")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and resta
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
modules.json: 100%                               349/349 [00:00<00:00, 22.4kB/s]

config_sentence_transformers.json: 100%           116/116 [00:00<00:00, 6.73kB/s]

README.md:      10.5k/? [00:00<00:00, 519kB/s]

sentence_bert_config.json: 100%                   53.0/53.0 [00:00<00:00, 2.88kB/s]

config.json: 100%                                612/612 [00:00<00:00, 25.8kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downl
model.safetensors: 100%                          90.9M/90.9M [00:01<00:00, 456MB/s]

Loading weights: 100%                           103/103 [00:00<00:00, 239.39it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%                      350/350 [00:00<00:00, 9.50kB/s]

vocab.txt:      232k/? [00:00<00:00, 3.71MB/s]

tokenizer.json:  466k/? [00:00<00:00, 7.63MB/s]

special_tokens_map.json: 100%                    112/112 [00:00<00:00, 2.88kB/s]

config.json: 100%                               190/190 [00:00<00:00, 5.09kB/s]

Embedding model loaded successfully
```

```
# Sample dataset
documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data."
```

```
    natural language processing deals with text data. ,  
    "RAG combines retrieval and generation."  
]  
  
print("Dataset loaded successfully")
```

Dataset loaded successfully

```
# Create ChromaDB client  
client = chromadb.Client()  
  
# Create collection  
collection = client.create_collection(name="rag_collection")  
  
print("ChromaDB collection created successfully")
```

ChromaDB collection created successfully

```
# Store documents with embeddings  
for i, doc in enumerate(documents):  
    embedding = model.encode(doc).tolist()  
  
    collection.add(  
        ids=[str(i)],  
        documents=[doc],  
        embeddings=[embedding]  
    )  
  
print("Documents stored successfully in ChromaDB")
```

Documents stored successfully in ChromaDB

```
# User query  
query = "What is Artificial Intelligence?"  
  
# Convert query into embedding  
query_embedding = model.encode(query).tolist()  
  
print("Query embedding created successfully")
```

Query embedding created successfully

```
# Retrieve relevant documents  
results = collection.query(  
    query_embeddings=[query_embedding],  
    n_results=2  
)  
  
print("Retrieved Documents:")  
for doc in results['documents'][0]:  
    print("-", doc)
```

Retrieved Documents:
- Artificial Intelligence is the simulation of human intelligence.
- Machine Learning is a subset of AI.

```
# Generate final answer using retrieved context
context = " ".join(results['documents'][0])

answer = f"Based on retrieved data: {context}"

print("\nFinal Answer:")
print(answer)
```

```
Final Answer:
Based on retrieved data: Artificial Intelligence is the simulation of human intelligence. Machine Learning is a subset of AI.
```